Article Type: Description (see below for more detail)

From Silent Automation to Conversational Intelligence: Empowering Smart Homes with LLM-Based Agents

Dhyego Tavares, Erlon Almeida, Enio Santana, Frederico Durao, Gustavo Bittencourt, Maycon Peixoto, Cassio Prazeres, Ivan Machado, Eduardo Santana de Almeida, *Federal University of Bahia, Brazil*

Abstract—Smart home systems have evolved from simple automation to AI-driven ecosystems. Nevertheless, user interaction often remains limited to rigid interfaces. This paper presents ECLIPS (Environmental Command Language Intelligent Processing System), an extension of a previously proposed Smart Home Orchestration Framework for a more intelligent and responsive environment. ECLIPS introduces a Two-Layer Modular Hierarchical Multi-Agent Architecture that integrates Large Language Model (LLM) agents as a semantic interpretation layer, while preserving established predictive models based on LSTM (Long Short-Term Memory) and MLP (Multi-Layer Perceptron) networks. The proposed approach enables natural language interaction, contextual information retrieval, and transparent explanation of system behavior without altering the underlying automation logic. Evaluated through a design science–oriented case study within an industrial smart home project, results demonstrate improved user understanding, flexibility, and control. Combining predictive machine learning with agentic LLM architectures enhances usability, transparency, and adaptability in smart home ecosystems.

Introduction

Smart home environments have transitioned from simple remote-controlled systems to sophisticated ecosystems driven by Artificial Intelligence (AI). In our previous work [1], [2], [3], we proposed a data-driven orchestration framework that uses LSTM and MLP models to learn user routines and generate automated recommendations. Although the framework demonstrated high accuracy in predicting behavior within the 'Company A' context, the interaction model presented a limiting factor. Communication between the human and the smart home was constrained to a binary paradigm: the system pushed a notification (e.g., suggesting turning off a light), and the user's role was reduced to a passive supervisor, limited to 'accepting' or 'rejecting' the action via a mobile interface. While this approach ensures validation, it fails to capture nuance in human intent, account for context, or explain the reasoning behind a decision, ultimately creating a barrier to a seamless user experience.

Concurrently, the field of AI has undergone a significant transformation with the emergence of Large Language Models (LLMs) and Generative AI agents [4]. These technologies have fundamentally altered human-machine communication, enabling systems to process and understand natural language with remarkable fluency. By leveraging advanced Natural Language Processing (NLP), modern AI agents can interpret complex intentions and maintain conversational context over time [5]. Recent studies have already applied LLM agents to conversational tasks, interactive recommendations, elderly care, and general assistance [6], [7], [8], [9]. This evolution offers a robust alternative to traditional graphical interfaces, suggesting a future where interacting with technology shifts from issuing discrete commands to conversing with context-aware assistants.

In this paper, we propose ECLIPS (Environmental Command Language Intelligent Processing System), an evolution of the previously proposed Smart Home

XXXX-XXX © 2021 IEEE
Digital Object Identifier 10.1109/XXX.0000.0000000

Orchestration Framework [1]. We introduce a Two-Layer Modular Hierarchical Multi-Agent Architecture designed to act as an advanced semantic interpretation layer. It is essential to emphasize that the underlying predictive architecture (LSTM and MLP) remains essentially unchanged and continues to serve as the engine for routine-based recommendation.

In this novel approach, the multi-agent system bridges the gap between raw system logs and human understanding. By adding a conversational interface alongside static panels, users gain granular insight into the environment, verify actions, locate devices within the spatial layout, and track state history, thereby providing transparency without altering the established predictive logic.

Research Methodology

Our research followed the principles of design science, a methodology that guides the development of purposeful technological solutions grounded in real-world challenges [10]. Building on the artifacts developed in earlier project phases, initially focused on device coordination and management policies, we broadened the solution to include a new component: a Generative AI–based Agent Module. This addition shifts the focus toward more intuitive, human-oriented interaction within the innovative environment (see the "System Architecture Evolution" section).

In alignment with design science's requirement for rigorous validation [10], the enhanced framework was investigated through a renewed case study embedded in the Smart Universe Project. Rather than solely optimizing automation, this study explores a refined research question: To what extent can LLM-enabled agents improve the user's ability to interact with the system, specifically accessing information, issuing complex commands, and maintaining seamless control over the smart home ecosystem?

Context Selection

Our investigation is part of the Smart Universe Project, a four-year research collaboration involving Company A and researchers from Applied Artificial Intelligence Lab (LIAA). This interdisciplinary effort brought together twelve researchers from LIAA, organized into four specialized teams:

- 1. **Recommendation Systems Team:** Composed of experts in recommendation systems.
- 2. **Software Engineering Team:** Comprised of researchers with expertise in software architecture, reuse for highly configurable systems, and testing.
- 3. **Infrastructure Team:** Made up of specialists in hardware, communication protocols, and IoT device modeling.
- 4. **DevOps Team:** Formed by experts in system deployment.

Additionally, five project managers acted as intermediaries between the research teams and ten industry practitioners from Company A.

Data Collection

The study draws on two primary data sources: (1) records of user inputs submitted to the agents and the corresponding outputs they generated, and (2) notes collected during demonstration sessions with selected experimental users.

Data collection spans the entire project development cycle, from January 2024 to December 2025. As an overview of the project's workload distribution, researchers typically participated in two weekly meetings, two biweekly meetings with senior researchers, and two annual showcase sessions with Company A. The estimated workload amounts to approximately 1,400 hours per researcher per year, dedicated to the development of ten major activities, each subdivided into more fine-grained tasks. However, due to a non-disclosure agreement, all project documents and source code must remain confidential.

Data Analysis

First, we analyzed the datasets and artifacts generated natively throughout the Agentic Module's development lifecycle. By examining these internal execution logs and system-generated data, we described its architecture. Based on these operational outputs, we present four case scenarios to optimize user usability and interaction with the system. These scenarios demonstrate the practical applicability of agents in a smart home system and provide a comprehensive view of their outcomes.

In addition, we reviewed documents and meeting notes to summarize the challenges and critical insights related to Agentic insertion in an AI-based smart home system. The documents describe the investigation of available devices and sensors to derive case scenarios for one or multiple users in the smart home when interacting with agents. From this information, we also identified future directions for contributing to the broader landscape of agent applications in smart home technologies.

System Architecture Evolution

The proposed architecture represents an evolution of the previous smart home framework [1], transitioning into a distributed, multi-agent ecosystem driven by Generative AI. The original architecture used a Core Module for event orchestration and a Smart Module based on LSTM and MLP neural networks. This structure processed sequential data to learn routines and generate automatic scene recommendations for users via a mobile application. Now the new system is organized into five distinct yet interconnected modules, as presented in Figure 1:

The central hub of the architecture is the *Core Module*, which functions as the primary orchestrator. It is responsible for monitoring events and managing data acquisition through external API endpoints. The *Core Module* explicitly handles the 'Converter Protocols Payload,' thereby ensuring interoperability by standardizing data flow between internal logic modules and external cloud services.

To handle predictive intelligence, the **Smart Module** retains its hybrid neural network foundation. It uses a combination of Long Short-Term Memory (LSTM) networks and Multilayer Perceptron (MLP) networks [2]. The LSTM component processes sequential data to capture temporal patterns in user behavior, while the MLP operates on these features to generate specific recommendations.

For human-in-the-loop interaction, the **Visualization Module** provides an interface for monitoring devices and system performance. It allows administrators to visualize critical data, such as energy consumption, while providing end-users with a link to provide feedback on automated actions.

The system interacts with the physical world through the Infrastructure Layer. This consists of a Third-Party Cloud that provides the SaaS architecture, SDKs, and APIs necessary for connectivity. This cloud layer manages direct communication with the Devices (sensors, plugs, and actuators) and executes the commands orchestrated by the internal modules.

A significant advancement in this architecture is the introduction of the Multi-Agents Module. Powered by GenAI (Generative AI), this module adds an agentic layer capable of advanced reasoning and collaboration. Crucially, to mitigate the strict latency constraints of voice interfaces, the Planner AI employs a Selective Model Routing mechanism. This ensures that high-priority, low-complexity commands (actuation) are processed by optimized, low-latency models, while reserving high-parameter models for complex reasoning and diagnostic queries. The Multi-Agents Module also interacts with the Core and **Smart Module**s to interpret complex contexts, potentially using the LSTM+MLP's probabilistic outputs to inform more dynamic, autonomous decision-making strategies that go beyond static rule-based systems.

The evolved Smart Home Orchestration Framework extends the foundational architecture established in our previous work [1], preserving the *Core Module* and **Smart Module** as the backbone for event orchestration and behavioral prediction. In this architecture, the LSTM and MLP networks remain the exclusive engines responsible for learning user patterns and generating probabilistic automation scenarios.

Hybrid Orchestration Topology

The pivotal architectural advancement presented in this work is the integration of the ECLIPS (Environmental Command Language Intelligent Processing System) module. Crucially, this Multi-Agent component is architected to operate in parallel with the existing **Visualization Module** (Mobile Application), rather than replace it (see Figure 1).

In this updated topology, ECLIPS functions as a parallel interface optimized for contextual retrieval and semantic interpretation. While the Mobile Application remains the primary mechanism for rapid manual device actuation and binary feedback on recommendations, the agentic layer interprets the state of the home environment. By interfacing directly with the *Core Module*'s data streams, the agents can query historical logs and real-time metadata to resolve natural-language queries about the "who, when, how, and where" of device interactions. This lateral configuration allows the system to significantly enhance the user's situational awareness, clarifying, for instance, which user triggered a specific scene or the precise timestamp of an activation, without disrupting the deterministic logic of the predictive models or introducing latency to the graphical interface's quick-access controls.

Multi-Agent Architecture

The operational flow of the proposed architecture follows a structured pipeline that converts unstructured natural language into deterministic actions or contextual explanations. As illustrated in Figure 2, the process comprises four distinct stages:

1. Input Processing and Intent Acquisition (A-B): The interaction begins when the User (A) issues a voice command or query. The Alexa (B) device serves as the primary audio interface, converting the spoken input into a text string via Automatic Speech Recognition (ASR). Unlike traditional integrations, where Alexa

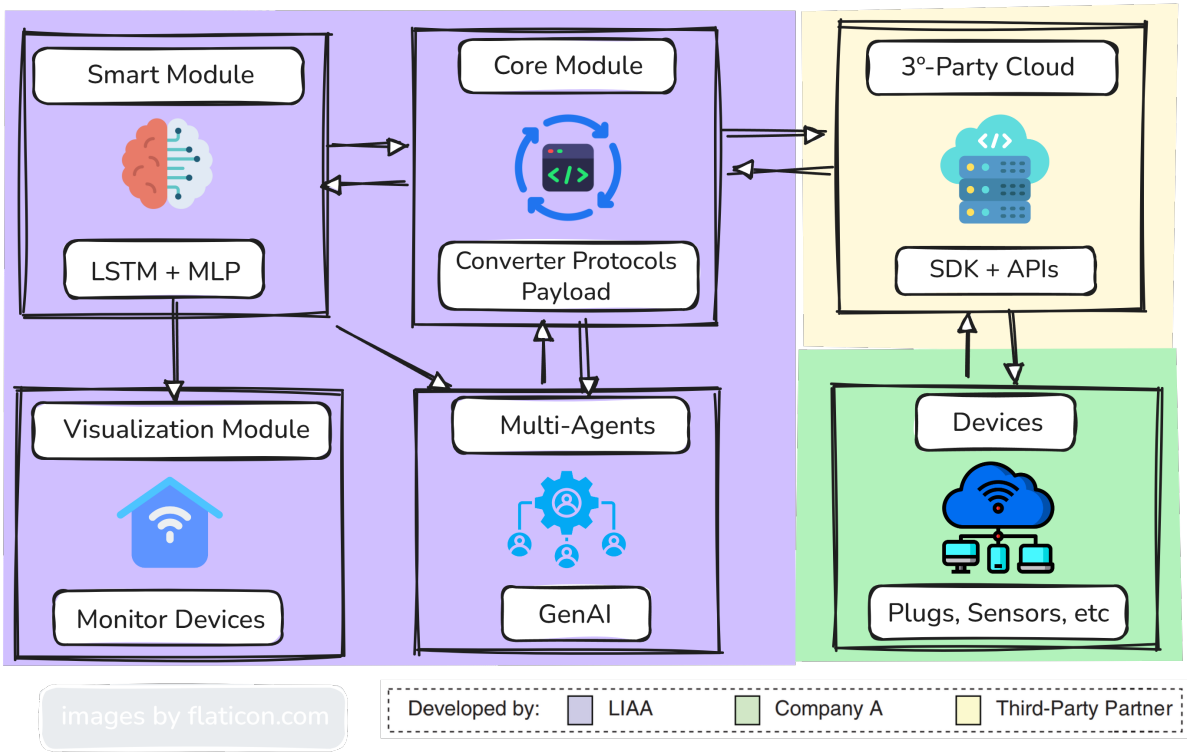

FIGURE 1. Agentic architecture

executes the logic directly, here it functions solely as an input/output bridge, forwarding the raw transcribed text to the ECLIPS Supervisor.

2. Semantic Orchestration (C): The text input is received by the Planner AI (C), which acts as the system's Supervisor Agent. Utilizing an LLM, the Planner AI analyzes the user's intent to determine the nature of the request. It performs a semantic routing decision:

- If the intent is identified as a query about past events or system status (e.g., "Who turned on the living room light?"), The task is delegated to the Recorder AI (D).
- If the intent is identified as a command to alter the state of the environment (e.g., "Turn on the office air conditioner"), the task is delegated to the JSON AI (F).

3. Specialized Execution (Layer 2): Depending on the routing decision, one of the Worker Agents takes over:

- **Context Retrieval (D-E):** The **Recorder AI (D)** accesses the Memory Store (E), a database containing the historical logs of all state changes and user interactions. It retrieves the relevant metadata and formulates a natural language answer, which is sent back to Alexa as a "House Info Response".
- **Command Synthesis (F-G):** The JSON AI (F) is responsible for actuation. To prevent hallucinations regarding device names or supported capabilities, it utilizes a Retrieval-Augmented Generation (RAG) tool connected to the Devices Register (G). This register acts as a source of truth for the available hardware schema. The agent then synthesizes a precise JSON payload containing the `device_id`, `action`, and parameters.

4. Asynchronous Actuation and Feedback Loop (H-K): The generated payload is published to the Broker (H), which acts as an asynchronous message bus.

- **Actuation:** The Broker forwards the request to the Protocol Converter (J), which translates the generic JSON into the specific API calls required by the Third Party Cloud (K) to trigger the device physically.
- **State Persistence:** Simultaneously, the Broker pushes the event data ("Actions Register") to the

Memory Store (E), ensuring that the Recorder AI has up-to-date context for future queries.

- **Feedback:** Upon successful execution, a status code is returned to the JSON AI, which triggers an "Action Confirmation" message back to the user via Alexa.

Architecture Design Discussion

The design of the ECLIPS architecture addresses four critical challenges in LLM-based home automation: hallucination rates in payload generation, system extensibility, hardware abstraction, and model latency control.

1. Hallucination Reduction via Specialization: A known limitation of monolithic LLM agents is the tendency to hallucinate device capabilities or syntax when managing broad contexts [11], [12], [13]. By implementing a hierarchical separation of concerns, we mitigate this risk. The Planner AI focuses exclusively on semantic intent classification, free from the constraints of code syntax. Conversely, the JSON AI is constrained to a specific execution scope. By grounding the JSON AI with RAG access to the Devices Register, the system ensures that generated command payloads strictly adhere to the valid schema of available devices, significantly increasing execution reliability.

2. Modularity and Scalability: The Supervisor-Worker topology ensures that the system remains open for extension but closed for modification. New automation domains, such as a "Music Controller AI" or "Security Guard AI", can be integrated by simply attaching new specialized Worker Agents to the hierarchy. This allows the system to scale its capabilities without retraining or fundamentally altering the prompt engineering of the central **Planner AI**, preserving the stability of the core orchestration logic.

3. Protocol Independence and Abstraction: The architecture enforces a strict decoupling between the agentic reasoning layer and the physical infrastructure [14], [15]. By utilizing an asynchronous **Broker** (Fig.2-H) and a **Protocol Converter** (Fig.2-J), the agents operate at a high level of abstraction, manipulating generic JSON objects rather than dealing with specific communication standards (e.g., code specifications from third-party cloud API illustrated in Fig.2-K). This isolation ensures that changes in hardware protocols do not propagate to the AI models, enhancing the maintainability of the software ecosystem.

4. Balanced Inference and Latency Management: The architecture directly addresses the trade-off between reasoning accuracy and real-time response latency. By integrating Selective Model Routing into the Planner AI, the system ensures that performance-critical functions (actuation) use optimized models for low latency (solving challenge 1). At the same time, data-intensive, logically complex queries leverage higher-parameter models to improve precision (addressing challenge 4). This hybrid approach stabilizes the system's reliability across diverse user requests.

Smart Home Application

In addition to the physical orchestration scenarios focused on comfort and energy efficiency, the platform's evolution with the integration of the Agents Module has enabled the exploration of a new dimension of interactivity: Conversational Investigation.

While the **Visualization Module** presents aggregated metrics, such as energy consumption and carbon footprint, through graphical dashboards in the mobile application, the Agents Module serves as a natural language interface to the history of events monitored by the *Core Module*. This capability transforms raw system logs, ranging from presence detection to device state changes, into auditable, explainable information for end users.

Consequently, the smart home shifts from being merely a command executor to becoming an active source of insights, allowing residents to trace the causality of automated actions (e.g., "why did the light turn on?") and monitor security and usage patterns. To validate this semantic information retrieval capability, we defined four additional scenarios that demonstrate the interaction between the user and the home's data history:

Scenario #1 **Home Security Insight**: The goal is to increase the user's sense of security by providing natural language access to entry logs and presence history.

User Inquiry: "Alexa, who was the last person to enter the house?"

- **1 - Agent Processing:** The Agent receives the voice command and identifies the intent
- **2 - Data Retrieval:** The Agent queries the *Core Module*'s event history for the most recent state change in the door lock or entrance motion sensor.
- **3 - Identity Resolution:** The system correlates the timestamp with the user credentials used in the mobile app or keypad.
- **4 - Response:** "The last person to enter was John Doe at 18:45 today."

Scenario #2 **Energy Consumption Diagnosis:**

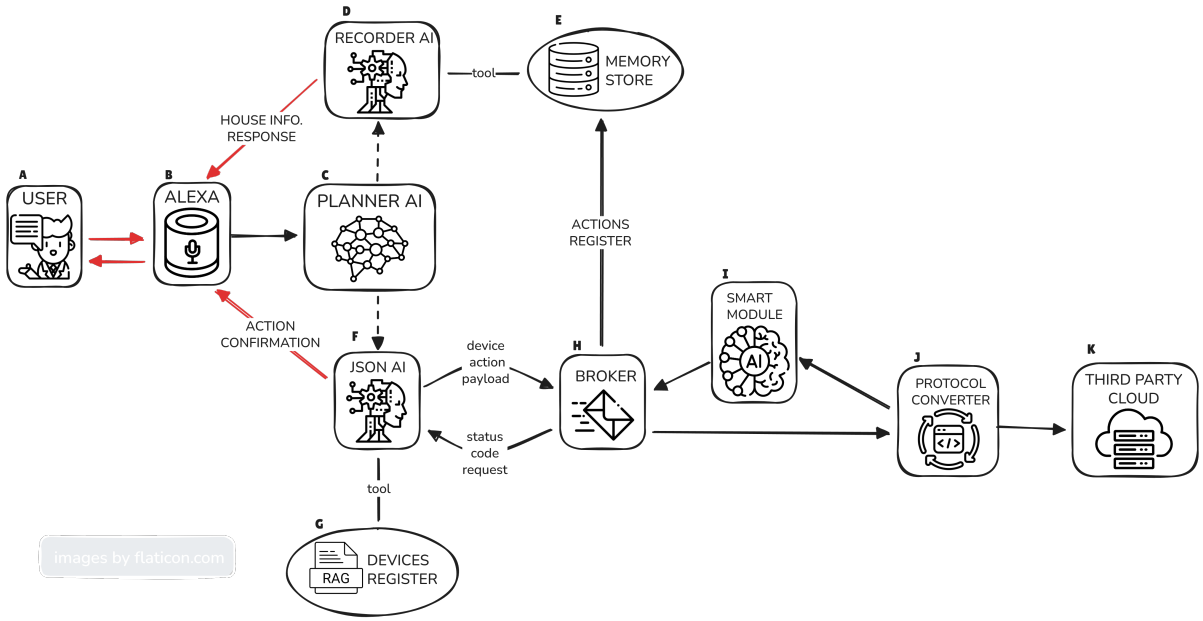

FIGURE 2. The ECLIPS Architecture: A Two-Layer Modular Hierarchical Multi-Agent System.

The goal is to offer granular transparency regarding energy usage, allowing the user to identify specific offenders without analyzing complex charts.

User Inquiry: "Which device consumed the most energy this week?"

- **1 - Agent Processing:** The Agent interprets the timeframe (last 7 days) and the metric (energy consumption in kWh).
- **2 - Data Retrieval:** The Agent requests the aggregated consumption data from the database, similar to the calculation used in the **Visualization Module**.
- **3 - Identity Resolution:** The system ranks devices based on accumulated scores.
- **4 - Response:** "The Air Conditioner in the Office was the highest consumer, responsible for 45% of the total energy usage this week."

Scenario #3 **Action Traceability:** The goal is to clarify the context of automated actions or manual interventions, resolving confusion about why a device is in a certain state.

User Inquiry: "Why is the living room light on?"

- **1 - Agent Processing:** The Agent checks the current state of the Light Bulb in the Living Room.
- **2 - Causality Check:** The Agent looks for the latest trigger event associated with that device ID in the *Core Module* logs.
- **3 - Differentiation:** It distinguishes if the action was triggered by a specific user (via App), a sensor (Motion Sensor), or an automation rule (**Smart Module** recommendation).
- **4 - Response:**"The living room light was turned on by the Motion Sensor detecting presence 5 minutes ago." OR "It was turned on manually by Olivia Smith via the app."

Scenario #4 **Routine & Occupancy Summary:** The goal is to provide a holistic view of the home's activity, summarizing complex sensor data into a digestible report for the user.

User Inquiry: "Give me a summary of the office usage today."

- **1 - Agent Processing:** The Agent aggregates data from Motion Sensors, Plugs (PC connection), and Air Conditioners located in the Office room.
- **2 - Causality Check:** The system calculates the total duration of occupancy based on time intervals.
- **3 - Response:** "The office was occupied for 6 hours today. The Air Conditioner ran for 4 hours at an average of 24℃, and the PC plug was active from 09:00 AM to 05:00 PM."

Limitation and Challenges

The introduction of the Agents Module, designed to operate laterally to the **Visualization Module**, has significantly enhanced user interaction by enabling human-like conversation via LLMs and voice assistants (e.g., Alexa). However, shifting from a purely app-based interface to a conversational ecosystem has introduced specific technical and usability challenges distinct from hardware and sensor limitations previously identified.

1. Latency Constraints in Voice Interfaces: A critical challenge in orchestrating voice commands involves the strict time-bound constraints imposed by third-party voice platforms. Interfaces such as Alexa require synchronous responses within a rigid window (approximately 8 seconds) to avoid timeouts. The processing pipeline, which encompasses speech-to-text transcription, LLM inference for intent extraction, and communication with the *Core Module* for execution, must be rigorously optimized. Deep reasoning models often exceed this latency threshold, resulting in synchronization failures in which a home action may be successfully executed by the *Core Module*. Still, the user's verbal confirmation fails because the voice provider terminates the connection.

2. User Expectation Alignment (Chatbot vs. Facilitator): The integration of LLMs introduces ambiguity regarding the agent's purpose. Users often conflate intelligent assistants with generative chatbots. However, the Agents Module is not designed for open-ended dialogue or general knowledge queries; it is strictly a facilitator for smart home access and device control. This dissonance can lead to user frustration when the system fails to process conversational interactions that fall outside the automation scope. This requires precise system prompt engineering to limit the agent's capabilities for the user clearly. To mitigate the user expectation challenge, the Planner AI implements a specialized 'Guard Prompt' strategy. This ensures that the agent's identity remains strictly that of a smart home facilitator, preventing it from engaging in open-ended dialogue. By incorporating Contextual Redirection, off-topic queries are not met with an error but with a gentle prompt to the agent's core capabilities, significantly reducing cognitive dissonance between a general chatbot and a specialized home agent.

3. Inference Complexity and Instruction Following: Although LLMs demonstrate high comprehension capabilities, translating natural language into structured actions for the smart home encounters barriers when requests involve high logical complexity. Orchestration requests that contain nested conditions, multi-step sequences, or ambiguous device references are not always handled correctly. The model's capacity to decompose intricate instructions into valid API calls for the *Core Module* is not always deterministic. This variability can result in unexpected actions or system inertia when encountering valid but structurally complex commands, potentially confusing the user about the system's reliability.

Conclusion

This work introduced the ECLIPS (Environmental Command Language Intelligent Processing System), an evolution of the Smart Home Orchestration Framework. We proposed a Two-Layer Modular Hierarchical Multi-Agent Architecture that serves as an advanced semantic interpretation layer and an evolution of our previous work. The multi-agent LLM architecture proposed in this work moves beyond the narrow paradigm of accepting recommendations toward more humanized interaction with the home and its devices, shifting from issuing discrete commands to conversing with context-aware assistance.

The evolution to ECLIPS demonstrates that merging established predictive models (LSTM+MLP) with Generative AI agents creates a smart home infrastructure that is more **intuitive, transparent, and responsive**. In particular, this enables practical advancements across essential domains: (1) automated safety and risk prevention, where continuous monitoring of people, devices, and structural conditions allows the system to detect anomalies, anticipate hazards, and trigger protective actions; (2) personalized comfort, as environmental configurations such as lighting, temperature, and soundscapes are dynamically adapted to user presence, preferences, and daily routines; (3) functional and proactive automation, with multi-device coordination that not only executes user commands but also predicts needs, eliminates repetitive tasks, and optimizes resource consumption; and (4) ambient support for health and well-being, including air-quality control, sleep-aware lighting policies, and smart assistance for vulnerable users through behavioral insights.

Future research will further optimize the Selective Model Routing mechanism to mitigate the latency constraints inherent in third-party voice interfaces, ensuring that complex reasoning remains within the required 8-second response window. We aim to enhance the system's instruction-following capabilities to more reliably handle nested conditions and multi-step sequences that currently present deterministic challenges. Additionally, we plan to exploit the architecture's modularity by integrating new specialized Worker Agents, such as a "Music Controller AI" or a

"Security Guard AI," to broaden the ecosystem's functional scope. Finally, we intend to conduct longitudinal studies on proactive automation specifically focused on health and well-being, exploring how behavioral insights can better support vulnerable users through ambient assistance.

ACKNOWLEDGMENTS

This work was financed by Fundação de Amparo à Pesquisa e Extensão (FAPEX) under grant number 391/2022, Fundação de Amparo a Pesquisa do Estado da Bahia (FAPESB) grants BOL0188/2020 and PIE0002/2022, and Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) grant 312195/2021-4.

REFERENCES

1. L. Martins, D. Campos, J. Mota, D. Tavares, J. Pereira, M. Oliveira, D. Boaventura, D. Correa, E. Ferreira, G. Pinto *et al.*, "A case study of smart home development," *IEEE Software*, 2024.
2. D. C. da Silva, D. Robson Dantas Boaventura, M. dos Santos Oliveira, J. Pereira Santos Junior, E. Ferreira da Silva, E. S. de Almeida, C. V. Prazeres, I. do Carmo Machado, M. Leone Maciel Peixoto, G. Bittencourt Figueiredo *et al.*, "Evaluating multi-label machine learning models for smart home environments," *Software: Practice and Experience*, 2025.
3. D. Campos, L. Martins, J. Mota, D. Tavares, J. Pereira, M. Oliveira, D. Boaventura, D. Correa, E. Ferreira, G. Pinto *et al.*, "Designing, implementing, and testing ai-oriented smart home applications: Challenges and best practices," in *European Conference on Software Architecture*. Springer, 2024, pp. 83–99.
4. W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, vol. 1, no. 2, 2023.
5. L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
6. X. Huang, J. Lian, Y. Lei, J. Yao, D. Lian, and X. Xie, "Recommender ai agent: Integrating large language models for interactive recommendations," *ACM Transactions on Information Systems*, vol. 43, no. 4, pp. 1–33, 2025.
7. K. Kim, H. Song, J. Ryu, C. Oh, and B. Suh, "Bleacherbot: Ai agent as a sports co-viewing partner," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–31.
8. Y. Guo, R. Wang, Z. Huang, T. Jin, X. Yao, Y.-L. Feng, W. Zhang, Y. Yao, and H. Mi, "Exploring the design of llm-based agent in enhancing self-disclosure among the older adults," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–17.
9. M. Tamascelli, O. Bunch, B. Fowler, M. Taeb, and A. Cohen, "Academic advising chatbot powered with ai agent," in *Proceedings of the 2025 ACM Southeast Conference*, 2025, pp. 195–202.
10. A. Hevner and S. Chatterjee, *Design research in information systems: theory and practice*. Springer Science & Business Media, 2010, vol. 22.
11. Z. Duan and J. Wang, "Enhancing multi-agent consensus through third-party llm integration: Analyzing uncertainty and mitigating hallucinations in large language models," in *2025 8th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*, 2025, pp. 2222–2227.
12. Z. Zhan, E. Haihong, and M. Song, "Leveraging large language model for enhanced text-to-sql parsing," *IEEE Access*, vol. 13, pp. 30 497–30 504, 2025.
13. X. Lin, Y. Ning, J. Zhang, Y. Dong, Y. Liu, Y. Wu, X. Qi, N. Sun, Y. Shang, K. Wang, P. Cao, Q. Wang, L. Zou, X. Chen, C. Zhou, J. Wu, P. Zhang, Q. Wen, S. Pan, B. Wang, Y. Cao, K. Chen, S. Hu, and L. Guo, "Llm-based agents suffer from hallucinations: A survey of taxonomy, methods, and directions," 2025. [Online]. Available: https://arxiv.org/abs/2509.18970
14. C. Fleming, V. Pandey, L. Muscariello, and R. Kompella, "A layered protocol architecture for the internet of agents," 2025. [Online]. Available: https://arxiv.org/abs/2511.19699
15. R. F. D. Rosario, K. Krawiecka, and C. S. de Witt, "Architecting resilient llm agents: A guide to secure plan-then-execute implementations," 2025. [Online]. Available: https://arxiv.org/abs/2509.08646

DHYEGO TAVARES is a researcher and Ph.D. student in computer science at the Federal University of Bahia, 40170-110 Salvador, Brazil. His research interests include software engineering, software quality, software reuse, and neuroscience for software engineering. Received an M.A. in computer science from the Federal University of Bahia. Contact him at dhyegocruz@ufba.br.

ERLON ALMEIDA is a researcher and M.Sc. student in Computer Science at the Federal University of Bahia,

40170-110 Salvador, Brazil. His research interests include software engineering, software quality, software testing, and AI for software engineering. Contact him at erlon.almeida@ufba.br.

ENIO G. SANTANA Jr. is a researcher and M.Sc. student in Computer Science at the Federal University of Bahia, 40170-110 Salvador, Brazil. His research interests include software engineering, software architecture, and AI for software engineering. Contact him at enio.garcia@ufba.br

FREDERICO ARAÚJO DURÃO is an associate professor at the Institute of Computing at the Federal University of Bahia, 40170-110 Salvador, Brazil, and a senior researcher and the project leader of the RecSys Research Group in Brazil, Federal University of Bahia, 40170-110 Salvador, Bahia. His research interests include information systems, recommender systems, and the semantic web. Durão completed his postdoctoral research at the Insight Centre for Data Analysis at University College Cork, Ireland, in 2016/2017, and obtained his Ph.D. in computer science from Aalborg University, Denmark, in 2012. Contact him at fdurao@ufba.br

GUSTAVO BITTENCOURT FIGUEIREDO is an associate professor at the Institute of Computing at the Federal University of Bahia, 40170-110 Salvador, Brazil, and is the leading researcher of the Intelligent Optical Networking Lab (ION Lab). His research interests include the planning, dimensioning, and optimization of optical and wireless networks. Figueiredo received a Ph.D. in computer science from the State University of Campinas. He is a Senior Member of IEEE. Contact him at gustavobf@ufba.br.

MAYCON PEIXOTO is an associate professor at the Institute of Computing at the Federal University of Bahia, 40170-110 Salvador, Brazil. His research interests include resource placement, management, and scheduling in cloud, edge, and fog computing. Peixoto received a Ph.D. in computer science from the University of São Paulo, Brazil. Contact him at maycon.leone@ufba.br.

CASSIO PRAZERES is an associate professor at the Institute of Computing at the Federal University of Bahia, 40170-110 Salvador, Brazil. His research interests include the Internet of Things, microservices, fog/edge computing, and edge artificial intelligence. Prazeres received a Ph.D. in computer science from the University of São Paulo. He cofounded the WISER Research Group. He is a Senior Member of IEEE. Contact him at prazeres@ufba.br.

IVAN MACHADO is an adjunct professor at the Institute of Computing at the Federal University of Bahia, 40170-110 Salvador, Brazil, and the head of the Applied Research in Software Engineering Lab (ARIES Lab). His research interests include software testing, empirical software engineering, applied machine learning, and mining software repositories. Machado received a Ph.D. in computer science from the Federal University of Bahia. He is a member of the IEEE, the Association for Computing Machinery (ACM), and the Brazilian Computer Society. Contact him at ivan.machado@ufba.br.

EDUARDO SANTANA DE ALMEIDA is an associate professor at the Institute of Computing at the Federal University of Bahia, 40170-110 Salvador, Brazil, and the head of the Reuse in Software Engineering Labs (RiSE Labs). His research interests include SE4AI, AI4SE, software reuse, software product lines, and empirical software engineering. Almeida received a Ph.D. in computer science from the Federal University of Pernambuco. He is a senior member of the Association for Computing Machinery and a Senior Member of IEEE. Contact him at eduardo.almeida@ufba.br.